

Markdown do projeto

Conteúdo do Documento de Resumo

Documento de Handoff & Resumo de Contexto - Projeto Estação de Sensores ESP32

****Status do Projeto:**** Funcional (Comunicação ESP-NOW + Servidor Web Assíncrono com LittleFS + HTML/CSS/JS Decouplados)

1. Visão Geral e Objetivo

Desenvolvimento de um sistema de monitoramento de sensores distribuído utilizando placas ESP32. O sistema consiste em módulos ****Emissores**** (que leem sensores como DHT11 e transmitem via ESP-NOW) e um módulo ****Receptor/Central**** (que recebe os dados via rádio, exibe no display local OLED, hospeda uma página Web local no LittleFS e fornece uma API JSON em tempo real via Wi-Fi para smartphones/navegadores).

1.1 Objetivo

Chegar a um protótipo tipo maquete com um sistema aplicável em que poderemos acionar um secador de cabelos via site hospedado em host, causando elevação de temperatura, e forçando o sistema de exaustão (ventilador) trazendo a temperatura de volta ao normal, com acionamento e desligamento automático.

2. Hardware e Infraestrutura

- ****Placas:**** ESP32 LoRa LyliGo (TTGO T-Display / DevKit v1) - 2 unidades
 - interface para ESP32 LoRa REV2.0/2022 com fonte de alimentação integrada 127vac~240vav-5vdc 0.6A - 2 unidades
- ****Mapeamento de Portas COM (Hub USB com Fonte Externa 5v 3A):****
 - ****COM3:**** Placa Receptora (Central Web)
 - ****COM4:**** Placa Emissora (Sensor DHT11)
- ****Periféricos & Sensores:****
 - Display OLED I2C (SSD1306): Pinos SDA=21, SCL=22 - cada unidade ESP32 carrega um Oled
 - Sensor DHT11 (na placa emissora) 1 unidade
 - Relé de estado sólido FOTEK SSR-25DA 25A 3-32vdc/24-380vac com dissipadores/fixadores de trilho DIN TS35 - 2 unidades
 - placa de relé HW-316 com 4 relés independentes JQC3F-05vdc-c 250vac 10A
 - 4 mini protoboards
 - 1 secador de cabelos residencial de 3500W 127v
 - 1 ventilador residencial de 50W 127v
- ****Rede / Wi-Fi:****
 - Modo Wi-Fi do Receptor: `WIFI\_AP\_STA` (Canal 1 fixo)
 - SSID da Rede Local: `ESP32\_Estacao` | Senha: `12345678`
 - IP de Acesso: `192.168.4.1`
 - ****Recursos extras disponíveis no laboratório****
 - Multímetro simples de uso geral para medições de componentes, continuidade e Vac Vdc
 - Roteador Wi-Fi residencial `NILSON 2.4`

- Senha `81111270in`
- Assinatura Hostgator M anual

3. Arquitetura de Software e Decisões Técnicas

- 1. **Ambiente de Desenvolvimento:**** VS Code + PlatformIO.
- 2. **Separação de Ambientes (`platformio.ini`):****
 - Uso da diretiva `build\_src\_filter` para compilar apenas o arquivo relevante em cada placa sem necessidade de renomear extensões.
 - Isolamento de dependências de bibliotecas (`lib\_deps`) por ambiente (Emissor não carrega servidor Web).
- 3. **Sistema de Arquivos (`LittleFS`):****
 - Arquivos web armazenados na pasta `data/` do projeto e gravados na Flash do ESP32 via target `uploadfs`.
 - Estrutura Web totalmente modularizada e desacoplada:
 - `data/index.html` (Estrutura da página)
 - `data/style.css` (Estilização)
 - `data/script.js` (Lógica AJAX com `fetch()` assíncrono a cada 2s com cache-busting)
- 4. **Comunicação e Servidor:****
 - ****Emissor -> Receptor:**** ESP-NOW (envio de struct de dados com temperatura, umidade e contador de pacotes).
 - ****Receptor -> Cliente Web:**** Servidor assíncrono (`ESPAsyncWebServer`) servindo arquivos estáticos e endpoint `/dados` fornecendo JSON.

4. Estrutura de Pastas do Projeto

```
MEU_PROJETO/
├── platformio.ini # Configuração dos ambientes emissor e receptor
├── data/ # Arquivos do Servidor Web (LittleFS - Receptor)
│   ├── index.html # Estrutura HTML
│   ├── style.css # Estilo visual
│   └── script.js # Atualização dinâmica (AJAX fetch)
├── src/
│   ├── emissor.cpp # Código Leve: Leitura do DHT11 + Envio ESP-NOW
│   └── receptor.cpp # Código Central: ESP-NOW + OLED + WebServer + LittleFS
```

5. Arquivos de Configuração e Código Atual

5.1. `platformio.ini`

```
[env]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
board_build.filesystem = littlefs
```

CONFIGURAÇÃO DA PLACA EMISSORA

```
[env:ttgo_emissor]
upload_port = COM4
monitor_port = COM4
build_src_filter = +<\*> -<receptor.cpp> -<main.cpp>
lib_deps =
adafruit/DHT sensor Library
adafruit/Adafruit Unified Sensor
adafruit/Adafruit GFX Library
adafruit/Adafruit SSD1306
```

CONFIGURAÇÃO DA PLACA RECEPTORA

```
[env:ttgo_receptor]
upload_port = COM3
monitor_port = COM3
build_src_filter = +<\*> -<emissor.cpp> -<main.cpp>
lib_deps =
adafruit/Adafruit GFX Library
adafruit/Adafruit SSD1306
https://github.com/me-no-dev/ESPAsyncWebServer.git
https://github.com/me-no-dev/AsyncTCP.git
```

```
### 5.2. `src/emissor.cpp`
```

```
#include <esp_now.h>
#include <esp_wifi.h>
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>
```

```
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
```

```
#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
```

```
#define PINO_AQUECEDOR 23
#define PINO_RETORNO_ENERGIA 34
```

```
unsigned long tempoInicioAquecedor = 0;
bool aquecedorAtivo = false;
const unsigned long DURACAO_AQUECEDOR = 30000; // 30s
```

```
uint8_t macReceptor[] = {0xA0, 0xDD, 0x6C, 0x67, 0xC6, 0x34}; // MAC do Receptor
```

```
typedef struct struct_mensagem {
int contador;
float temperatura;
```

```
float umidade;
bool energiaOk;
} struct_mensagem;

typedef struct struct_comando {
bool acionarAquecedor;
} struct_comando;

struct_mensagem dadosEnvio;
struct_comando comandoRecebido;
esp_now_peer_info_t peerInfo;

int contadorPacotes = 0;
unsigned long ultimoEnvio = 0;

void AoReceberComando(const uint8_t *mac, const uint8_t *incomingData, int len) {
// Força o acionamento direto sem validar o tamanho estrito da struct
digitalWrite(PINO_AQUECEDOR, HIGH);
aquecedorAtivo = true;
tempoInicioAquecedor = millis();
Serial.println(">>> AQUECEDOR LIGADO VIA ESP-NOW! <<<");
}

void setup() {
Serial.begin(115200);

// Inicialização do OLED no Emissor
Wire.begin(21, 22);
display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
display.setTextColor(WHITE);
display.setTextSize(1);
display.clearDisplay();
display.setCursor(0, 0);
display.println("PLACA A (EMISSOR)");
display.display();

// Configuração e inicialização do DHT11
pinMode(DHTPIN, INPUT_PULLUP);
dht.begin();
delay(2000); // Estabilização do sensor

pinMode(PINO_AQUECEDOR, OUTPUT);
digitalWrite(PINO_AQUECEDOR, LOW);
pinMode(PINO_RETORNO_ENERGIA, INPUT);

WiFi.mode(WIFI_STA);

esp_wifi_set_promiscuous(true);
esp_wifi_set_channel(1, WIFI_SECOND_CHAN_NONE);
esp_wifi_set_promiscuous(false);

if (esp_now_init() == ESP_OK) {
    esp_now_register_recv_cb(AoReceberComando);
    memcpy(peerInfo.peer_addr, macReceptor, 6);
```

```
        peerInfo.channel = 1;
        peerInfo.encrypt = false;
        esp_now_add_peer(&peerInfo);
    }

}

void loop() {
    // Desligamento automático temporizado
    if (aquecedorAtivo && (millis() - tempoInicioAquecedor >= DURACAO_AQUECEDOR)) {
        digitalWrite(PINO_AQUECEDOR, LOW);
        aquecedorAtivo = false;
    }

    if (millis() - ultimoEnvio >= 2000) {
        ultimoEnvio = millis();

        float temp = dht.readTemperature();
        float umid = dht.readHumidity();

        if (!isnan(temp) && !isnan(umid)) {
            dadosEnvio.temperatura = temp;
            dadosEnvio.umidade = umid;
        }

        contadorPacotes++;
        dadosEnvio.contador = contadorPacotes;
        dadosEnvio.energiaOk = digitalRead(PINO_RETORNO_ENERGIA);

        // Atualização do Display OLED local no Emissor
        display.clearDisplay();
        display.setCursor(0, 0);
        display.println("PLACA A (EMISSOR)");
        display.setCursor(0, 16);
        display.print("Temp: ");
        display.print(dadosEnvio.temperatura, 1);
        display.println(" C");
        display.setCursor(0, 32);
        display.print("Umid: ");
        display.print(dadosEnvio.umidade, 1);
        display.println(" %");
        display.setCursor(0, 48);
        display.print("Pacote enviado: #");
        display.println(dadosEnvio.contador);
        display.display();

        esp_now_send(macReceptor, (uint8_t *)&dadosEnvio, sizeof(dadosEnvio));
    }

}
```

5.3. `src/receptor.cpp`

```
#include <esp_now.h>
```

```
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Arduino.h>
#include <ESPAsyncWebServer.h>
#include <LittleFS.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
AsyncWebServer server(80);

uint8_t macEmissor[] = {0xA0, 0xDD, 0x6C, 0x75, 0x0E, 0x14}; // MAC do Emissor

// Struct idêntica à do Emissor
typedef struct struct_mensagem {
  int contador;
  float temperatura;
  float umidade;
  bool energiaOk;
} struct_mensagem;

typedef struct struct_comando {
  bool acionarAquecedor;
} struct_comando;

struct_mensagem dadosRecebidos;
struct_comando comandoEnvio;
esp_now_peer_info_t peerInfo;

void AoReceber(const uint8_t *mac_addr, const uint8_t *incomingData, int len) {
  if (len == sizeof(dadosRecebidos)) {
    memcpy(&dadosRecebidos, incomingData, sizeof(dadosRecebidos));

    display.clearDisplay();
    display.setCursor(0, 0);
    display.println("PLACA B (RECEPTOR)");
    display.setCursor(0, 16);
    display.print("Temp: ");
    display.print(dadosRecebidos.temperatura, 1);
    display.println(" C");
    display.setCursor(0, 32);
    display.print("Umid: ");
    display.print(dadosRecebidos.umidade, 1);
    display.println(" %");
    display.setCursor(0, 48);
    display.print("Pacote: #");
    display.println(dadosRecebidos.contador);
    display.display();
  }
}
```

```
void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.setTextColor(WHITE);
  display.setTextSize(1);

  if (!LittleFS.begin(true)) {
    Serial.println("Erro ao montar o LittleFS!");
  }

  WiFi.mode(WIFI_AP_STA);
  WiFi.softAP("ESP32_Estacao", "12345678", 1);

  if (esp_now_init() == ESP_OK) {
    esp_now_register_recv_cb(AoReceber);
    memcpy(peerInfo.peer_addr, macEmissor, 6);
    peerInfo.channel = 1;
    peerInfo.encrypt = false;
    esp_now_add_peer(&peerInfo);
  }

  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request) {
    request->send(LittleFS, "/index.html", "text/html");
  });
  server.on("/style.css", HTTP_GET, [](AsyncWebServerRequest *request) {
    request->send(LittleFS, "/style.css", "text/css");
  });
  server.on("/script.js", HTTP_GET, [](AsyncWebServerRequest *request) {
    request->send(LittleFS, "/script.js", "text/javascript");
  });

  server.on("/dados", HTTP_GET, [](AsyncWebServerRequest *request) {
    float t = isnan(dadosRecebidos.temperatura) ? 0.0 :
dadosRecebidos.temperatura;
    float h = isnan(dadosRecebidos.umidade) ? 0.0 : dadosRecebidos.umidade;

    String json = "{";
    json += "\"temperatura\":" + String(t, 1) + ",";
    json += "\"umidade\":" + String(h, 1) + ",";
    json += "\"contador\":" + String(dadosRecebidos.contador) + ",";
    json += "\"energia_ok\":" + String(dadosRecebidos.energiaOk ? "true" :
"false");
    json += "}";

    request->send(200, "application/json", json);
  });

  server.on("/ligar-aquecedor", HTTP_POST, [](AsyncWebServerRequest *request) {
    comandoEnvio.acionarAquecedor = true;
    esp_err_t result = esp_now_send(macEmissor, (uint8_t *)&comandoEnvio,
sizeof(comandoEnvio));

    if (result == ESP_OK) {
```

```
        request->send(200, "text/plain", "Comando enviado com sucesso");
    } else {
        request->send(500, "text/plain", "Erro ao enviar via ESP-NOW");
    }
});

server.begin();

}

void loop() {

### 5.4. Arquivos da Pasta `data/`

#### `data/index.html`

<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Painel ESP32</title>
    <link rel="stylesheet" type="text/css" href="style.css">
    <script src="script.js" defer></script>
</head>
<body>
    <h1>Estação de Sensores</h1>

    <div class="card">
        <p>Temperatura</p>
        <div class="valor"><span id="temp">--</span> °C</div>
    </div>

    <div class="card">
        <p>Umididade</p>
        <div class="valor"><span id="umid">--</span> %</div>
    </div>

    <div class="card">
        <h3>Controle do Aquecedor</h3>
        <button id="btnAquecedor" onclick="acionarAquecedor()">Ligar Aquecedor
(30s)</button>
        <div class="status-container">
            <span>Retorno de Energia:</span>
            <span id="ledStatus" class="led desligado"></span>
        </div>
    </div>

    <p>Pacotes recebidos: <b id="contador">0</b></p>

</body>
</html>
```



```
#### `data/style.css`
```

```
body {
font-family: Arial, sans-serif;
text-align: center;
margin-top: 30px;
background-color: #f4f4f9;
}
h1 {
color: #333;
}
.card {
background: white;
padding: 20px;
margin: 10px auto;
width: 260px;
border-radius: 10px;
box-shadow: 0 2px 5px rgba(0,0,0,0.2);
}
.valor {
font-size: 2em;
color: #007bff;
font-weight: bold;
}
button {
background-color: #28a745;
color: white;
border: none;
padding: 12px 20px;
font-size: 1em;
border-radius: 5px;
cursor: pointer;
transition: 0.3s;
}
button:disabled {
background-color: #6c757d;
cursor: not-allowed;
}
.status-container {
margin-top: 15px;
display: flex;
align-items: center;
justify-content: center;
gap: 10px;
}
.led {
width: 16px;
height: 16px;
border-radius: 50%;
display: inline-block;
background-color: #ccc;
}
.led.ligado {
background-color: #dc3545; /* Bolinha vermelha quando ativado */
}
```

```
box-shadow: 0 0 8px #dc3545;
}
.led.desligado {
background-color: #6c757d;
}

#### `data/script.js`

function atualizarDados() {
fetch('/dados?t=' + new Date().getTime(), { cache: 'no-store' })
.then(response => {
if (!response.ok) throw new Error('Erro na resposta');
return response.json();
})
.then(data => {
// Atualiza os valores na tela
document.getElementById('temp').innerText = data.temperatura;
document.getElementById('umid').innerText = data.umidade;
document.getElementById('contador').innerText = data.contador;

// Atualiza o LED de retorno de energia
const led = document.getElementById('ledStatus');
if (data.energia_ok) {
led.className = 'led ligado';
} else {
led.className = 'led desligado';
}
})
.catch(err => console.log('Aguardando dados...', err));
}

function acionarAquecedor() {
const btn = document.getElementById('btnAquecedor');
if (btn) btn.disabled = true;

fetch('/ligar-aquecedor', {
method: 'POST',
cache: 'no-store'
})
.then(response => {
console.log('Comando enviado com sucesso');
// Força uma atualização imediata do painel após o clique
atualizarDados();
})
.catch(err => console.log('Erro ao acionar:', err))
.finally(() => {
// Reabilita o botão após 1 segundo
setTimeout(() => {
if (btn) btn.disabled = false;
}, 1000);
});
}
```

```
// Garante o loop contínuo de atualização a cada 2 segundos
setInterval(atualizarDados, 2000);

// Executa a primeira leitura assim que a página carrega
document.addEventListener('DOMContentLoaded', atualizarDados);
```

6. Fluxo de Comandos e Gravação (PlatformIO CLI)

1. ****Upload do LittleFS (Receptor / COM3):****
`pio run -e ttgo\_receptor -t uploadfs`
2. ****Upload do Firmware do Receptor (COM3):****
`pio run -e ttgo\_receptor -t upload`
3. ****Upload do Firmware do Emissor (COM4):****
`pio run -e ttgo\_emissor -t upload`

7. Próximos Passos

- 5 - inserção de código para acionamento de ventilador, caso a temperatura ultrapasse um limite determinado, e desligue quando chegar à temp determinada
- 6 - criação de um banco de dados sql no provedor hostgator
- 7 - inserção de dados automaticamente (hora/temperatura/umidade) no banco de dados com PHP
- 8 - inserção de site para acesso global na hostgator tão simples quanto o site do LittleFS
- 9 - inserção de logo e relatorio de dados do banco sql no site
- 10 - teste de campo para verificação de funcionamento à distancia com powerbank alimentando o receptor, e teste via internet com smartfone
- 11 - configurar nova unidade esp32 LyLigo oled (que ainda será adquirida) para servir de gateway e aumentar alcance
- 12 - definir todas as necessidades de aplicações com seus respectivos sensores (apenas quando todos os passos anteriores forem consolidados)
- 13 - Refatorar o código C++ aplicando ****Programação Orientada a Objetos (POO)**** com classes dedicadas (`DisplayManager`, `WebServerManager`, `SensorManager`).
- 14 - Suporte a múltiplos transmissores.
- 15 - Sistema de verificação de tensão e corrente com sensor sct-013 com bias ou offset (usando entrada analógica ADC-Analog to Digital Converter). Definir 3 estados de verificação, desligado->corrente 0, normal, falha->corrente muito abaixo ou muito acima, objetivo é verificar não só liga/desliga, mas também aparelhos com mal funcionamento.
- 16 - Sistema de filtro de registro de dados em banco sql, para não registrar cada segundo de verificação.
- 17 - Implementar sistema com POO de forma a facilitar o recurso de instalação de novas unidades e código organizado e reutilizavel.

8. Informações gerais de parametros para IA auxiliar

- respostas diretas e sem abstrações.

- cada fase, e cada passo deve ser respeitado, só passamos adiante depois de: funcionamento confirmado, commit local e nuvem, incrementação de documentação do projeto.
- quando indicar o passo a passo de ligações eletrônicas, pode ser uma lista de itens curtos, pois tenho maior domínio.
- manter os números dos pinos descritos nos códigos, ou, avisar quando precisar redefinir algum em nova versão de código.

8.1 Lista de commits com descrições e respectivas datas

- Date: Mon Jul 27 12:49:25 2026 atualização de documentação
- Date: Sun Jul 26 22:18:42 2026
botaoHTML_AionaAquecedor30seg/retornoEnergiaConfirmaAcionamentoAcendeBolinhaHTML
- Date: Sat Jul 25 17:39:53 2026 Adição de documento resumo projeto
- Date: Sat Jul 25 17:13:46 2026 adiciona serv web assincrono com LittleFs (html,css,js)
- Date: Wed Jul 22 00:48:32 2026 inserção DHT11 sensor temperatura/umidade
- Date: Tue Jul 21 15:17:37 2026 Comunic Bidirecional base ESP-NOW OLED